

AI BlogNest API - Project Documentation

System Specification & Engineering Architecture

Project Leader: Harish. A

Development Team:

- Kamesh. R
 - Deepak. P
 - Manikandan. M
 - Shree. P
-

Executive Summary & System Overview

Executive Summary

The **AI BlogNest API** (ContentGen Infrastructure) is an enterprise-grade backend RESTful service engineered to power next-generation publishing platforms. Built on Node.js and Express.js with MongoDB as its core data store, the platform integrates automated content engine capabilities with robust access management to deliver scalable publishing operations.

System Overview

AI BlogNest decoupling separates computational content generation from client-side presentation, giving development teams standard APIs for content management, media routing, and automated semantic analysis.

Primary Platform Capabilities

- **Generative AI Pipeline:** Native integration with Google Gemini API models to deliver dynamic blog generation, structured FAQ extraction, context-aware WeatherWise summaries, and tailored FitTrack health recommendations.
 - **Role-Based Access Control (RBAC):** Strict permissions architecture enforcing authorization limits across four primary tiers: Admin, Editor, Author, and Reader.
 - **Stateless Security Stack:** Standardized JSON Web Token (JWT) handling combined with salted `bcrypt.js` hashing for credential protection.
 - **Production Reliability:** Centralized error handling, input sanitization, dynamic API rate limiting, and structured JSON log generation.
-

Scenario-Based Case Study

Background Context

Priya is a digital content operations lead who oversees a multi-niche publication network. Her team faced bottlenecks from disjointed management tooling, manual editing queues, and slow content drafting workflows.

Problem Statement

- **Fragmented Editing Pipelines:** High friction when managing multi-author draft queues, category assignments, and content taxonomy tagging across separate portals.
- **Analytics Blind Spots:** Low visibility into post performance metrics, reader reaction ratios, and real-time engagement data.
- **Publishing Delays:** Slow manual moderation for submitted reader comments and author pull requests.
- **Granularity Deficits:** Standard CMS options lacked fine-grained, role-based access limits for external contributors versus internal staff editors.

Technical Solution & Usage Workflow

AI BlogNest solves these challenges through unified REST endpoints managed by an internal permissions matrix:

- **Author Workflow:** Rapidly generates draft structures using Gemini AI endpoints, links uploaded media routes, and tracks post metrics through dedicated user dashboards.
- **Editor & Admin Workflow:** Reviews queue states, converts content statuses from **draft** to **published**, runs bulk taxonomy cleanup, and adjusts authorization levels.
- **Reader Interface:** Consumes public endpoints via CDN/frontend interfaces with low latencies while submitting interactive feedback through secure moderation endpoints.

Quantitative & Operational Outcomes

- **Reduced Delivery Cycles:** Draft-to-publication times dropped by utilizing automated AI outline generation.
- **Strict Validation Standards:** Automated payload checks cut data corruption issues across taxonomy trees.
- **Auditable Operations:** Enterprise-grade security protocols ensured zero privilege leakage during administrative state changes.

System Requirements & Hardware Specifications

Software Stack Requirements

Component	Minimum Requirement	Platform Purpose
Operating System	Linux / macOS / Windows 10+	Host operating system environment

Component	Minimum Requirement	Platform Purpose
Node.js	v16.0.0 or higher	Non-blocking asynchronous JavaScript runtime environment
npm	v8.0.0 or higher	Package dependency manager
Express.js	v4.18.0+	Core web application router framework
MongoDB	v6.0+ / MongoDB Atlas	Document-oriented NoSQL database system
Testing Suite	Postman / ThunderClient	API contract execution and route testing framework
IDE	Visual Studio Code	Primary development environment

Hardware Hardware Specifications

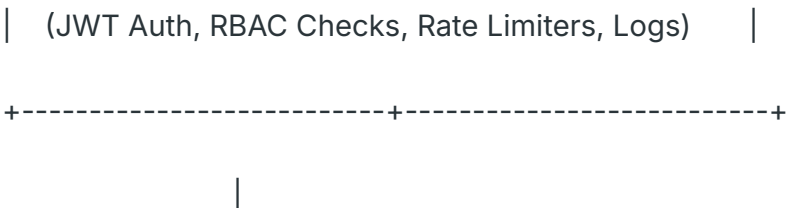
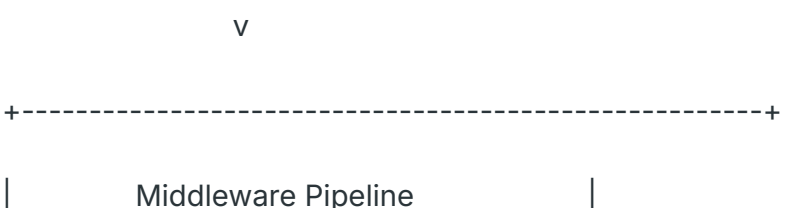
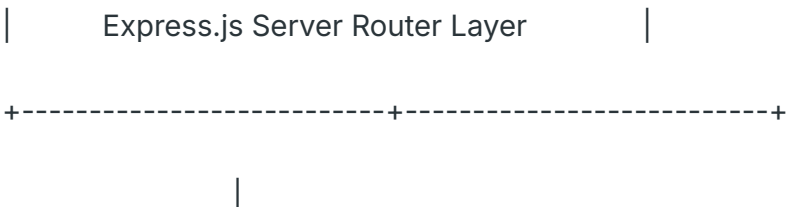
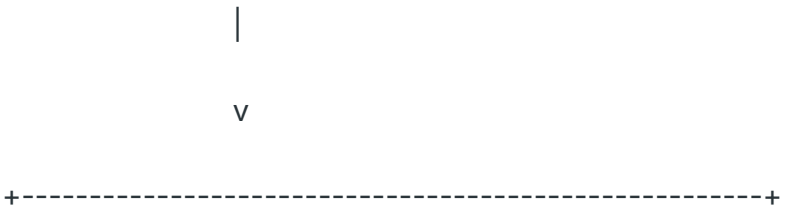
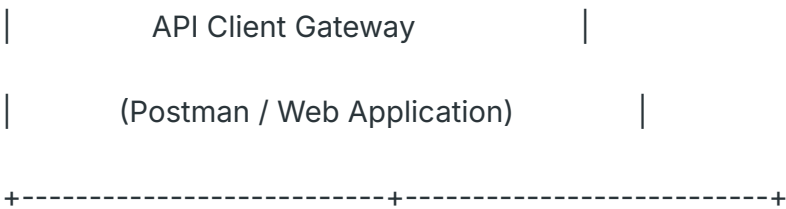
- **Processor:** Intel Core i5 (8th Generation or above), AMD Ryzen 5 series, or Apple Silicon ARM chips.
 - **System Memory (RAM):** 8 GB minimum; 16 GB recommended to support local concurrent MongoDB daemon execution and Node.js process debugging.
 - **Storage Capacity:** 1 GB of free disk space for repository source files, native binaries, and application logs.
-

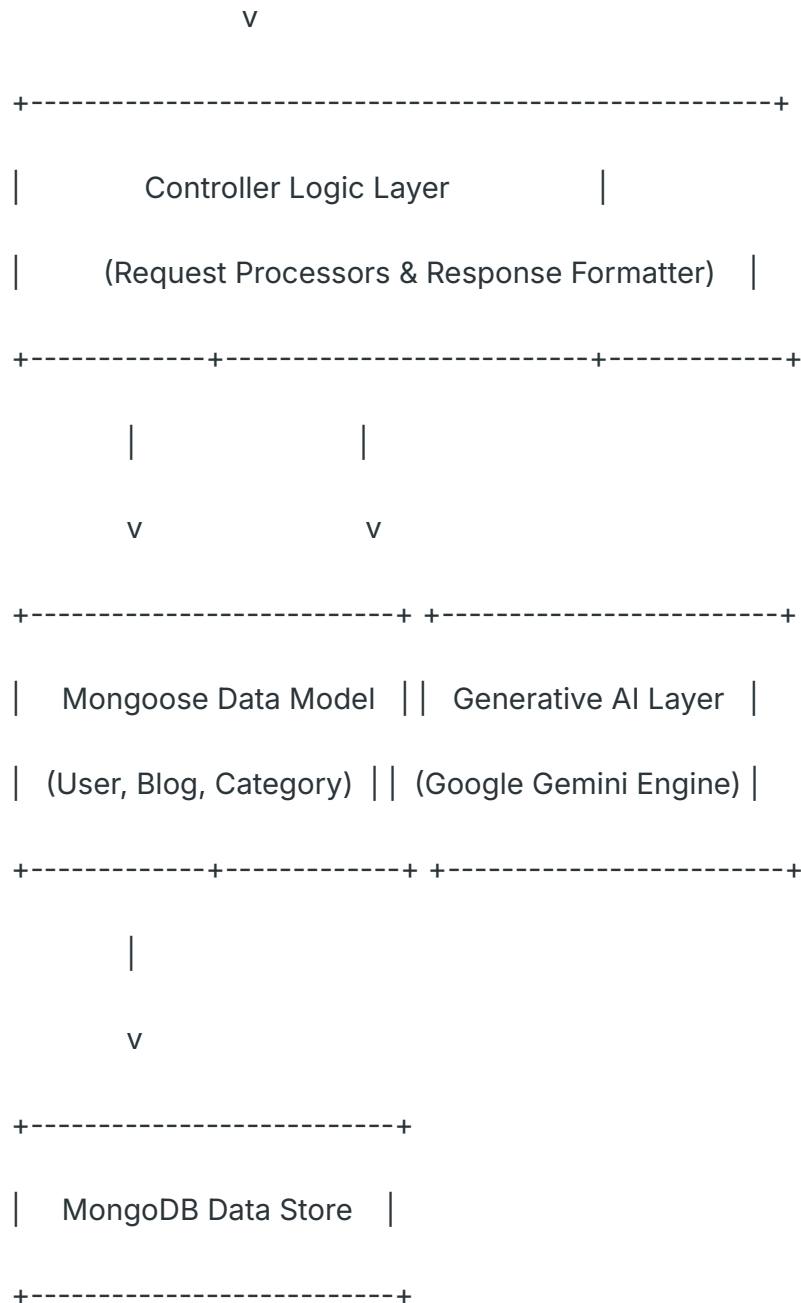
Project Architecture & MVC Design Pattern

Architecture Layer Breakdown

The AI BlogNest service uses a decoupled multi-tiered architecture to isolate operational responsibilities and maintain system

integrity: +-----+





Model-View-Controller Design Strategy

- **Model Layer:** Built with Mongoose schemas, defining strict data structures, field validations, hooks, and automated schema indexing for database operations.
- **View Layer (API Router Gateway):** Operates as a headless system. Routes map request URLs directly to controller functions, returning structured JSON payloads to clients.

- **Controller Layer:** Contains the application's core logic. Handles incoming requests, enforces business rules, calls database services, and communicates with external AI services.
-

Database Schema & Data Modeling

Database Entities & Attributes

User Entity

- `_id`: ObjectId (Primary Key, Auto-indexed)
- `name`: String (Required, Trimmed)
- `email`: String (Required, Unique, Lowercase Index)
- `password`: String (Required, Hashed with bcrypt)
- `role`: String (Enum: `admin`, `editor`, `author`, `reader`; Default: `reader`)
- `createdAt`: Timestamp

Blog Post Entity

- `_id`: ObjectId (Primary Key)
- `userID`: ObjectId (Foreign Key → User)
- `title`: String (Required)
- `slug`: String (Unique Index)
- `excerpt`: String
- `content`: String (Required)
- `coverImage`: String (URI Path)
- `status`: String (Enum: `draft`, `published`; Default: `draft`)
- `likes`: Number (Default: 0)
- `timestamps`: Real-time creation/modification markers

Category Entity

- `_id`: ObjectId (Primary Key)
- `name`: String (Required, Unique)

- `slug`: String (Unique Index)
- `description`: String
- `createdAt`: Timestamp

Comment Entity

- `_id`: ObjectId (Primary Key)
- `blogID`: ObjectId (Foreign Key → Blog Post)
- `userID`: ObjectId (Foreign Key → User)
- `content`: String (Required)
- `status`: String (Enum: `active`, `inactive`; Default: `active`)
- `createdAt`: Timestamp

Relationship Mapping

- **User to Blog Post**: \$1 : N\$ (An author creates multiple posts; each post links to one primary user).
 - **Category to Blog Post**: \$1 : N\$ (A category contains multiple posts; each post belongs to a primary category).
 - **Blog Post to Comment**: \$1 : N\$ (A blog post holds multiple comments).
 - **User to Comment**: \$1 : N\$ (A user posts multiple comments across the system).
 - **User to Blog Post Likes**: \$N : M\$ (Users like multiple posts; posts collect likes from multiple users).
-

User Flow & Privilege Matrix

End-to-End System Flow

1. **Client Request**: External clients dispatch HTTP payloads to targeted endpoints.
2. **Authentication Check**: The middleware inspects `Authorization: Bearer <token>` header flags.
3. **Stateless JWT Verification**: The cryptographic signature is evaluated against system keys.

- 4. **Context Injection:** Valid user details and system permissions attach directly to the request object (`req.user`).
- 5. **RBAC Validation:** Endpoint routing passes through role-based access checks.
 - **Standard Operations:** Executes standard blog reading, comment drafting, or AI-assisted content tools.
 - **Administrative Operations:** Grants access to user moderation, category updates, system-wide analytics, and role modifications.
- 6. **Controller & Service Processing:** Executed actions query the Mongoose ORM layer or external service APIs.
- 7. **Database Persistence:** Operations complete securely within the primary MongoDB database.

Privilege Matrix

Operation Category	Reader	Author	Editor	Admin
Read Public Posts & Comments	Yes ▾	Yes ▾	Yes ▾	Yes ▾
Submit New Comments	Yes ▾	Yes ▾	Yes ▾	Yes ▾
Create Draft Posts	No ▾	Yes ▾	Yes ▾	Yes ▾
Edit Owned Posts	No ▾	Yes ▾	Yes ▾	Yes ▾
Modify Unowned Posts	No ▾	No ▾	Yes ▾	Yes ▾
Manage System Categories	No ▾	No ▾	Yes ▾	Yes ▾
Delete User Accounts	No ▾	No ▾	No ▾	Yes ▾
Execute Admin Operations	No ▾	No ▾	No ▾	Yes ▾

Setup, Directory Structure & Server Setup

Directory Layout Architecture

ai-blognest-api/

```
├── src/
│   ├── config/
│   │   └── db.js
│   ├── controllers/
│   │   ├── aiController.js
│   │   ├── authController.js
│   │   └── blogController.js
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   └── errorMiddleware.js
│   ├── models/
│   │   ├── Blog.js
│   │   ├── Category.js
│   │   └── User.js
│   ├── routes/
│   │   └── aiRoutes.js
```

```
| | | — authRoutes.js
| | | — blogRoutes.js
| | — services/
| | — aiService.js
| — .env
| — app.js
| — package.json
| — server.js
```

Setup & Dependency Configuration

Initialize the repository runtime environment and install essential production dependencies:h

Initialize Node.js Package Workspace

```
npm init -y
```

Install Essential Production Dependencies

```
npm install express mongoose dotenv cors morgan jsonwebtoken bcryptjs
@google/generative-ai
```

```
### Main Application File (`app.js`)
```

```
``javascript
```

```
const express = require('express');

const cors = require('cors');

const morgan = require('morgan');

const errorHandler = require('./src/middleware/errorMiddleware');

const app = express();

// Global Middleware Config

app.use(express.json());

app.use(cors());

app.use(morgan('dev'));

// Routing Mount Points

app.use('/api/auth', require('./src/routes/authRoutes'));

app.use('/api/blogs', require('./src/routes/blogRoutes'));

app.use('/api/ai', require('./src/routes/aiRoutes'));

// Central Error Handler

app.use(errorHandler);

module.exports = app;
```

API Endpoints & Verification Matrix

Primary API Routes

1. Authentication Endpoints

POST **/api/auth/register**

- **Purpose:** Registers a new user account on the platform.
- **Access Level:** Public.
- **Payload Request:**

```
{  
  
  "name": "Jane Doe",  
  
  "email": "jane@example.com",  
  
  "password": "SecurePassword123!"  
}
```

- **Status Response:** 201 Created

POST **/api/auth/login**

- **Purpose:** Validates credentials and returns a signed JWT token.
- **Access Level:** Public.
- **Payload Request:**

```
{  
  
  "email": "jane@example.com",  
  
  "password": "SecurePassword123!"  
}
```

- **Status Response:** 200 OK (Returns **token** string).

2. Blog Management Endpoints

POST `/api/blogs`

- **Purpose:** Creates a new blog post entry linked to the active user.
- **Access Level:** Protected (`Author` / `Editor` / `Admin`).
- **Payload Request:**

```
{  
  
  "title": "Building Modern Scalable APIs",  
  
  "content": "Comprehensive deep dive into modern asynchronous backend design  
patterns...",  
  
  "category": "Technology"  
}
```

- **Status Response:** `201 Created`

GET `/api/blogs`

- **Purpose:** Retrieves a paginated list of published blog posts.
- **Access Level:** Public.
- **Status Response:** `200 OK`

3. AI-Powered Integration Endpoints

POST `/api/ai/generate-blog`

- **Purpose:** Generates a structured post draft using the Gemini AI engine.
- **Access Level:** Protected (`Author` / `Editor` / `Admin`).
- **Payload Request:**

```
{  
  
  "topic": "Microservices Design Patterns",  
  
  "category": "Software Architecture"
```

```
}
```

- **Status Response:** 200 OK

POST `/api/ai/summarize`

- **Purpose:** Produces concise summaries from longer text bodies.
- **Access Level:** Public / Protected.
- **Payload Request:**

```
{
```

```
"content": "Long form blog text goes here..."
```

```
}
```

- **Status Response:** 200 OK
-

Conclusion & Project Repositories

Documentation Summary

The **AI BlogNest API** demonstrates a secure backend architecture built on Node.js, Express.js, and MongoDB. By pairing traditional REST services with Google Gemini AI capabilities, the platform automates editorial workflows while maintaining granular access controls and reliable performance.

Project Repositories & Artifacts

All project artifacts, source repositories, testing suites, and video demonstrations can be accessed via the reference links below:

- **Source Code Repository:** [GitHub Repository Anchor](#)
- **API Demonstration Video:** [Watch Demonstration Video](#)
- **ThunderClient Workspace Export:** [Download Environment Specs](#)